



Normalisasi dan Standarisasi

Data pada Machine Learning

Dr. Jasman Pardede, S.Si., M.T.

Normalisasi

- Salah satu tantangan yang ada pada dataset adalah adanya atribut dengan unit, **rentang**, dan **skala yang berbeda**.
- Menerapkan algoritma machine learning ataupun data mining pada data dengan **rentang yang drastis** dapat memberikan **hasil yang kurang akurat**. Karena itu dibutuhkan proses normalisasi data.
- Normalisasi data adalah elemen dasar pada *machine learning* dan data mining untuk **memastikan record** pada dataset **tetap konsisten**.
- Normalisasi umumnya **diperlukan** apabila **atribut-atribut dari dataset** memiliki **skala atau rentang yang berbeda**.

Normalisasi (2)

- Normalisasi pada dasarnya adalah teknik **perubahan skala** yang mana kita merubah nilai dari data kedalam skala diantara **0–1** atau **antara -1 ke 1**.
- **Tujuan normalisasi** adalah mengubah nilai kolom **numerik** dalam himpunan data untuk menggunakan skala umum, tanpa mendistorsi perbedaan dalam rentang nilai atau kehilangan informasi.
- Normalisasi dilakukan pada data yang memang **mengikuti distribusi normal**
- Normalisasi memiliki jangkauan nilai 0–1

Data yang tidak standar

No	Nama	Gaji	Lama Bekerja	Posisi
1	Aman	100,000	10	2
2	Budiman	78,000	7	4
3	Cintya	32,000	5	8
4	Deddy	55,000	6	7
5	Efendy	92,000	8	3
6	Fandy	120,000	15	1
7	Gandy	65,750	7	5

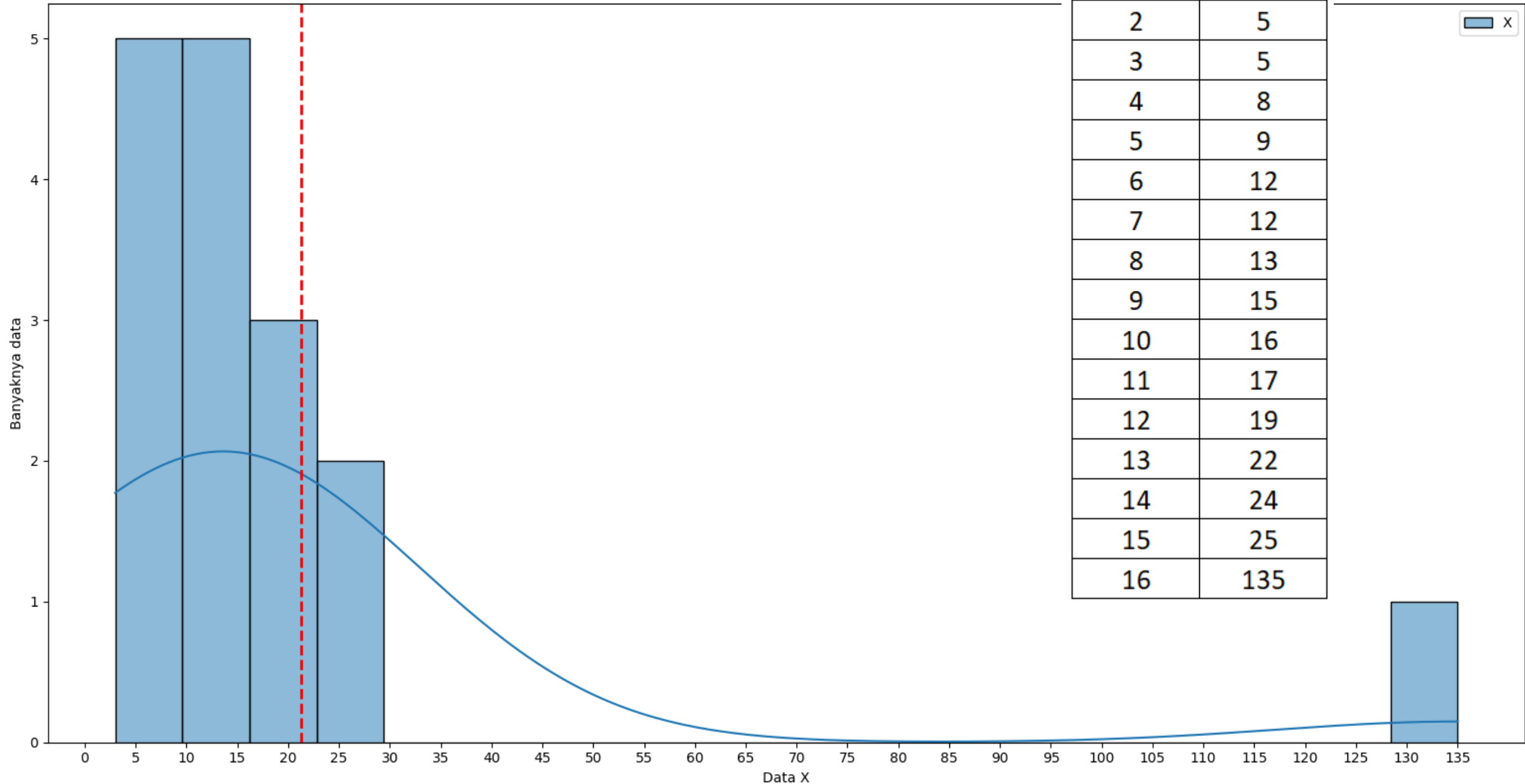
Teknik-Teknik Normalisasi Data

- Normalisasi Z-score
- Normalisasi Min-Max
- Normalisasi Decimal Scaling
- Normalisasi Sigmoid
- Normalisasi Softmax

No	Data
1	3
2	5
3	5
4	8
5	9
6	12
7	12
8	13
9	15
10	16
11	17
12	19
13	22
14	24
15	25
16	135

Data Original

Sebaran Data X sebelum normalisasi z score



Normalisasi z-score

$$x_{new} = \frac{x - \bar{x}}{\sigma} \quad (7.1)$$

dimana:

x_{new} : nilai baru hasil normalisasi data

$\bar{x}$: mean (nilai rata-rata)

σ : standar deviasi

x : nilai pengamatan x sebelum normalisasi

Normalisasi z-score (2)

No	Data
1	3
2	5
3	5
4	8
5	9
6	12
7	12
8	13
9	15
10	16
11	17
12	19
13	22
14	24
15	25
16	135

$$x_{1_new} = \frac{3 - 21.25}{30.08} = -0.607$$

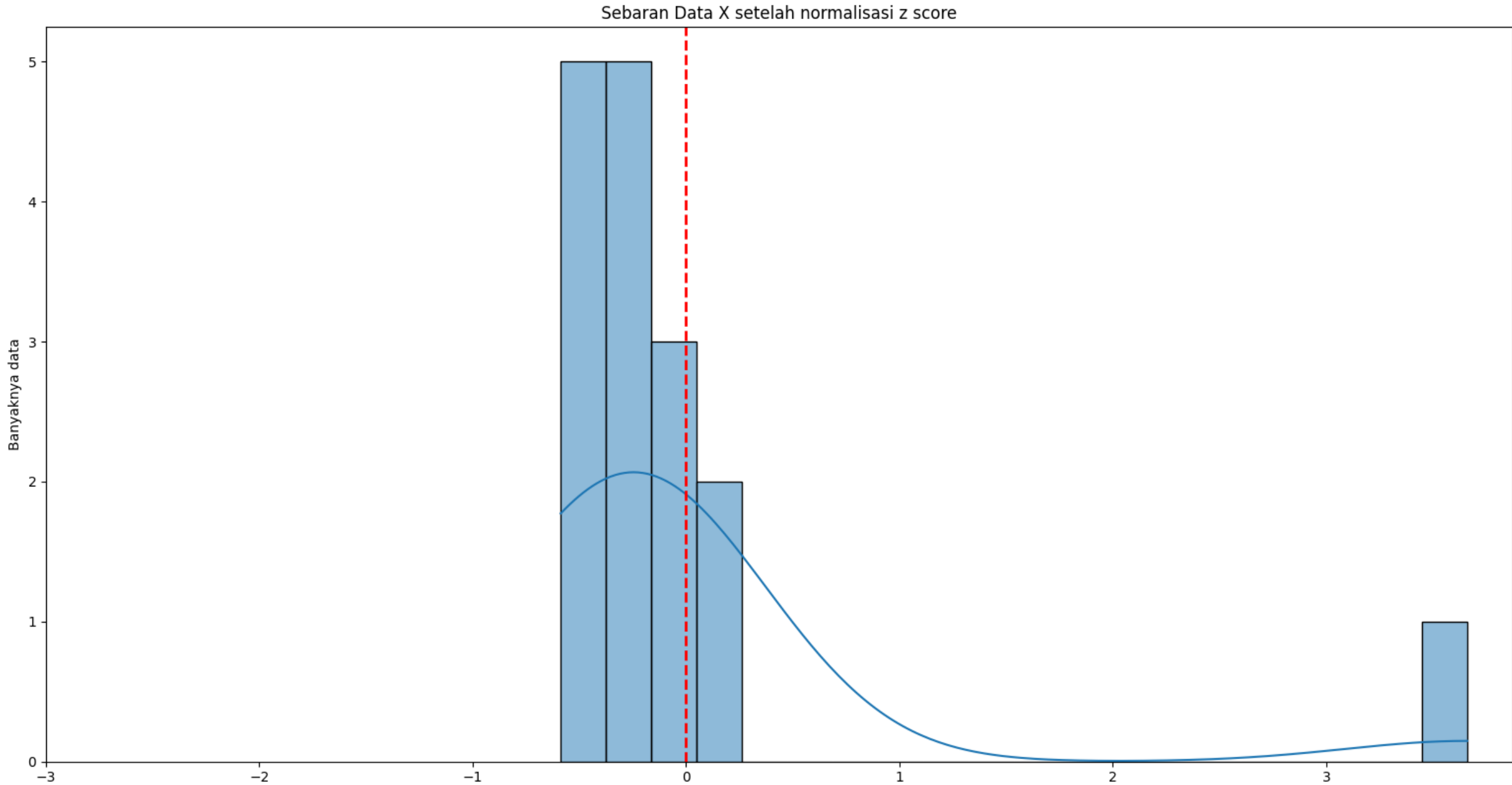
$$x_{2_new} = \frac{5 - 21.25}{30.08} = -0.540$$

No	X	X_{new}
1	3	-0.607
2	5	-0.540
3	5	-0.540
4	8	-0.440
5	9	-0.407
6	12	-0.308
7	12	-0.308
8	13	-0.274
9	15	-0.208
10	16	-0.175
11	17	-0.141
12	19	-0.075
13	22	0.025
14	24	0.091
15	25	0.125
16	135	3.782

Implementasi python

```
1 import matplotlib.pyplot as plt
2 import numpy as np
3 import seaborn as sns
4 import pandas as pd
5
6 data = [3, 5, 5, 8, 9, 12, 12, 13, 15, 16, 17, 19, 22, 24, 25, 135]
7 myData = pd.DataFrame(data)
8 myData.columns = ["X"]
9 print(myData)
10 sns.histplot(myData, bins=20, kde=True)
11 plt.axvline(np.mean(data), color='r', linestyle='dashed', linewidth=2)
12 plt.xticks(range(0,140,5))
13 plt.xlabel('Data X')
14 plt.ylabel('Banyaknya data')
15 plt.title(" Sebaran Data X sebelum normalisasi z score ")
16 plt.show()
17
18 mu = myData['X'].mean()
19 sigma = myData['X'].std()
20 data = (data - mu)/sigma
21 sns.histplot(data, bins=20, kde=True)
22 plt.xlabel('Z-score ')
23 plt.axvline(np.mean(data), color='r', linestyle='dashed', linewidth=2)
24 plt.ylabel('Banyaknya data')
25 plt.xticks(range(-3,4,1))
26 plt.title(" Sebaran Data X setelah normalisasi z score ")
27 plt.show()
```

Normalisasi Z-score



Normalisasi Min Max

$$x_{new} = \frac{x - x_{min}}{x_{max} - x_{min}} \quad (7.2)$$

dimana:

x_{new} : nilai baru hasil normalisasi data

x : nilai pengamatan x sebelum normalisasi

x_{min} : nilai absolut minimal dari x

x_{max} : nilai absolut maksimal dari x

Normalisasi Min Max (2)

No	Data
1	3
2	5
3	5
4	8
5	9
6	12
7	12
8	13
9	15
10	16
11	17
12	19
13	22
14	24
15	25
16	135

$$x_{1_new} = \frac{3 - 3}{135} = 0$$

$$x_{2_new} = \frac{5 - 3}{135} = 0.015$$

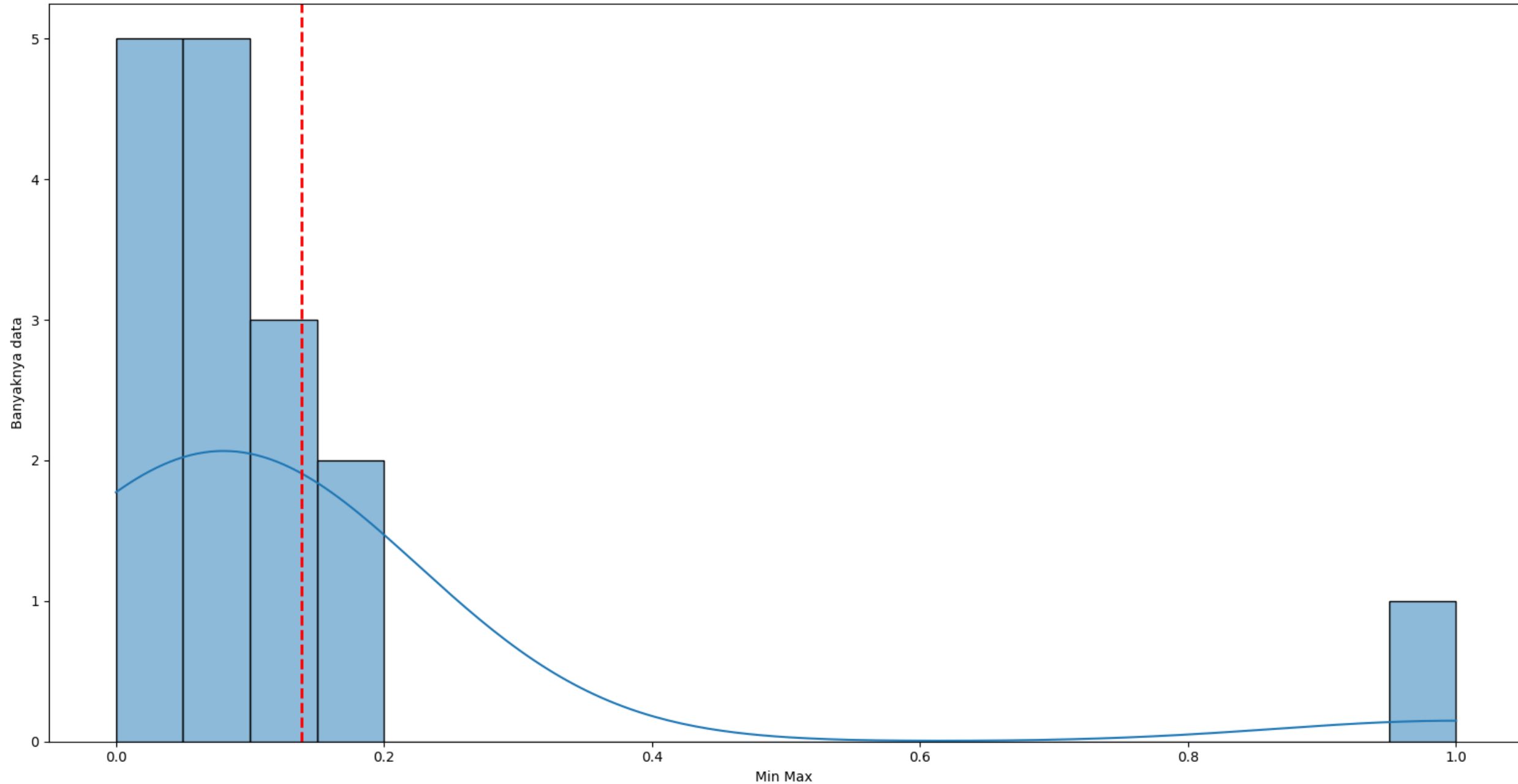
No	X	X_{new}
1	3	0.000
2	5	0.015
3	5	0.015
4	8	0.038
5	9	0.045
6	12	0.068
7	12	0.068
8	13	0.076
9	15	0.091
10	16	0.098
11	17	0.106
12	19	0.121
13	22	0.144
14	24	0.159
15	25	0.167
16	135	1.000

Implementasi python

```
1 import matplotlib.pyplot as plt
2 import numpy as np
3 import seaborn as sns
4 import pandas as pd
5
6 data = [3, 5, 5, 8, 9, 12, 12, 13, 15, 16, 17, 19, 22, 24, 25, 135]
7 myData = pd.DataFrame(data)
8 myData.columns = ["X"]
9 print(myData)
10 sns.histplot(myData, bins=20, kde=True)
11 plt.axvline(np.mean(data), color='r', linestyle='dashed', linewidth=2)
12 plt.xticks(range(0,140,5))
13 plt.xlabel('Data X')
14 plt.ylabel('Banyaknya data')
15 plt.title(" Sebaran Data X sebelum normalisasi z score ")
16 plt.show()
17
18 dt_min = np.min(data)
19 dt_max = np.max(data)
20 data = (data - dt_min)/(dt_max - dt_min)
21 sns.histplot(data, bins=20, kde=True)
22 plt.xlabel('Min Max ')
23 plt.axvline(np.mean(data), color='r', linestyle='dashed', linewidth=2)
24 plt.ylabel('Banyaknya data')
25 plt.title("Sebaran Data X setelah normalisasi min max ")
26 plt.show()
```

Min Max Normalisasi

Sebaran Data X setelah normalisasi min max



Normalisasi Skala Desimal

$$x_{new} = \frac{x}{10^j} \quad (7.3)$$

dimana:

x_{new} : nilai baru hasil normalisasi data

x : nilai pengamatan x sebelum normalisasi

j adalah bilangan bulat terkecil sehingga $\max(|x_{new}|) < 1$

Normalisasi Skala Desimal (2)

No	Data
1	3
2	5
3	5
4	8
5	9
6	12
7	12
8	13
9	15
10	16
11	17
12	19
13	22
14	24
15	25
16	135

➤ Max = 135, sehingga nilai $j = 3$.

$$x_{1_new} = \frac{3}{10^3} = \frac{3}{1000} = 0.003$$

$$x_{2_new} = \frac{5}{1000} = 0.005$$

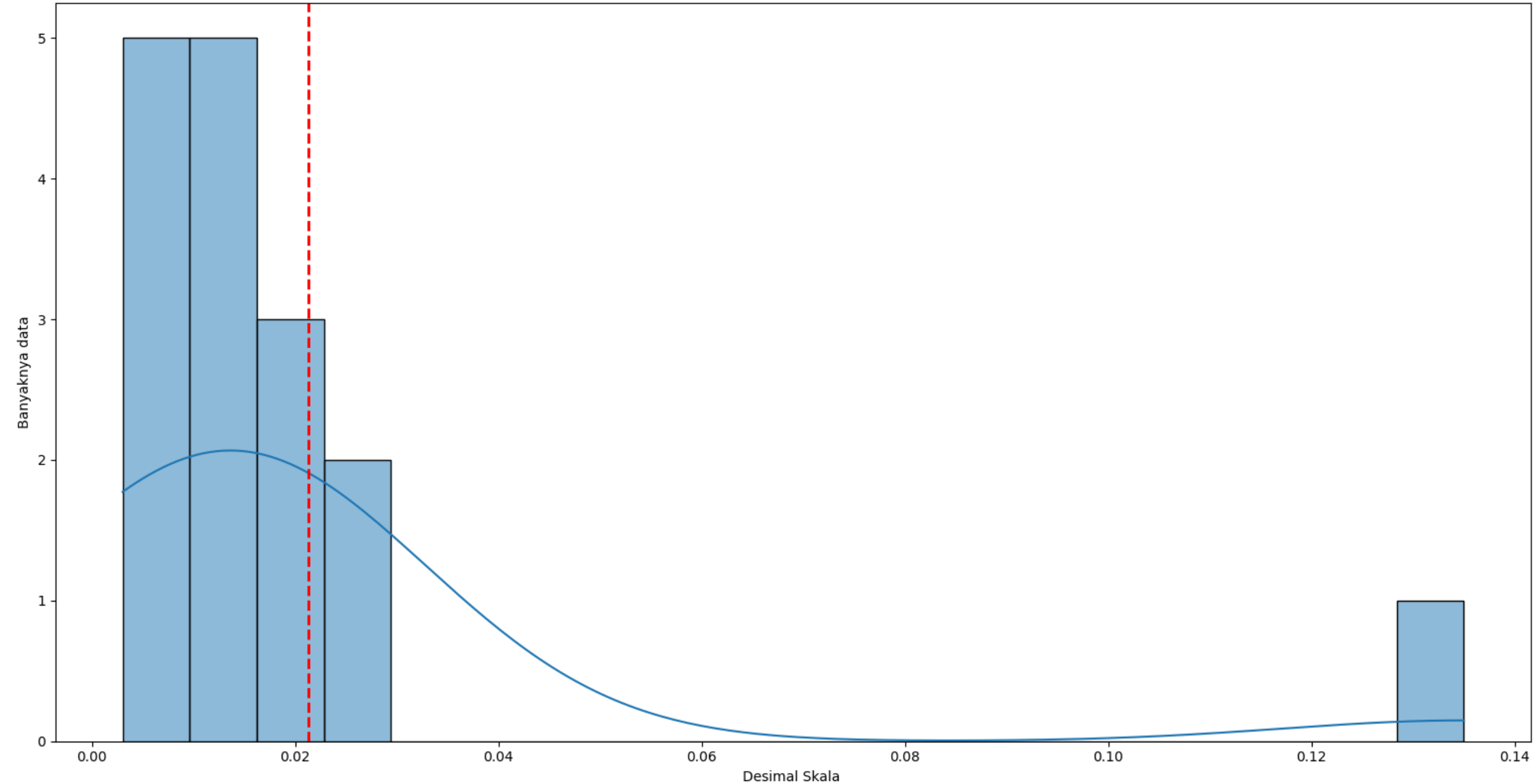
No	X	X_{new}
1	3	0.003
2	5	0.005
3	5	0.005
4	8	0.008
5	9	0.009
6	12	0.012
7	12	0.012
8	13	0.013
9	15	0.015
10	16	0.016
11	17	0.017
12	19	0.019
13	22	0.022
14	24	0.024
15	25	0.025
16	135	0.135

Implementasi python

```
1 import matplotlib.pyplot as plt
2 import numpy as np
3 import seaborn as sns
4 import pandas as pd
5
6 data = [3, 5, 5, 8, 9, 12, 12, 13, 15, 16, 17, 19, 22, 24, 25, 135]
7 myData = pd.DataFrame(data)
8 myData.columns = ["X"]
9 print(myData)
10 sns.histplot(myData, bins=20, kde=True)
11 plt.axvline(np.mean(data), color='r', linestyle='dashed', linewidth=2)
12 plt.xticks(range(0,140,5))
13 plt.xlabel('Data X')
14 plt.ylabel('Banyaknya data')
15 plt.title(" Sebaran Data X sebelum normalisasi z score ")
16 plt.show()
17
18 j = math.ceil(np.log10(np.max(np.abs(myData["X"]).to_numpy()))))
19 div_pow = math.pow(10, j)
20 data = (myData["X"].to_numpy())/(div_pow)
21 sns.histplot(data, bins=20, kde=True)
22 plt.xlabel('Desimal Skala ')
23 plt.axvline(np.mean(data), color='r', linestyle='dashed', linewidth=2)
24 plt.ylabel('Banyaknya data')
25 plt.title("Sebaran Data X setelah normalisasi Desimal skala ")
26 plt.show()
```

Desimal Skala

Sebaran Data X setelah normalisasi Desimal skala



Normalisasi Sigmoid

$$transpormasi_x = \frac{x - \bar{x}}{std} \quad (7.4)$$

$$x_{new} = \frac{1 - \exp(-transpormasi_x)}{1 + \exp(-transpormasi_x)} \quad (7.5)$$

dimana:

x_{new} : nilai baru hasil normalisasi data

$\bar{x}$: mean (nilai rata-rata)

σ : standar deviasi

x : nilai pengamatan x sebelum normalisasi

$transpormasi_x$: nilai z-score

Normalisasi Sigmoid (2)

No	Data
1	3
2	5
3	5
4	8
5	9
6	12
7	12
8	13
9	15
10	16
11	17
12	19
13	22
14	24
15	25
16	135

➤ Mean = 21.25

➤ Std = 30.08

$$Z_{1_score} = \frac{3 - 21.25}{30.08} = -0.607$$

$$Z_{2_score} = \frac{5 - 21.25}{30.08} = -0.540$$

$$x_{new} = \frac{1 - \exp(0.607)}{1 + \exp(0.607)} = -0.294$$

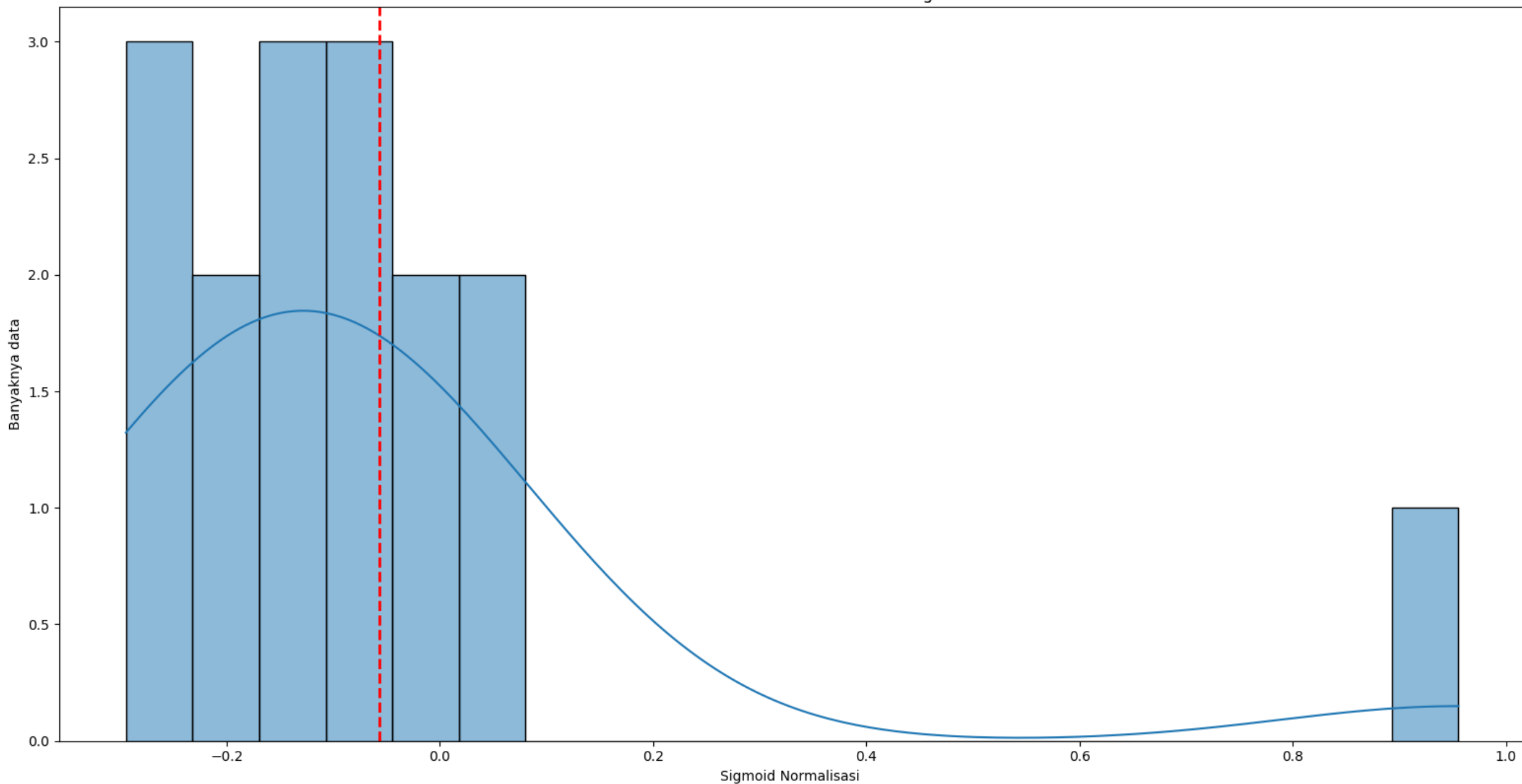
No	X	Z _{score}	X _{new}
1	3	-0.607	-0.294
2	5	-0.540	-0.264
3	5	-0.540	-0.264
4	8	-0.440	-0.217
5	9	-0.407	-0.201
6	12	-0.308	-0.153
7	12	-0.308	-0.153
8	13	-0.274	-0.136
9	15	-0.208	-0.104
10	16	-0.175	-0.087
11	17	-0.141	-0.071
12	19	-0.075	-0.037
13	22	0.025	0.012
14	24	0.091	0.046
15	25	0.125	0.062
16	135	3.782	0.955

Implementasi python

```
1 import matplotlib.pyplot as plt
2 import numpy as np
3 import seaborn as sns
4 import pandas as pd
5
6 data = [3, 5, 5, 8, 9, 12, 12, 13, 15, 16, 17, 19, 22, 24, 25, 135]
7 myData = pd.DataFrame(data)
8 myData.columns = ["X"]
9 print(myData)
10 sns.histplot(myData, bins=20, kde=True)
11 plt.axvline(np.mean(data), color='r', linestyle='dashed', linewidth=2)
12 plt.xticks(range(0,140,5))
13 plt.xlabel('Data X')
14 plt.ylabel('Banyaknya data')
15 plt.title("Sebaran Data X sebelum normalisasi z score ")
16 plt.show()
17
18 data = myData["X"].to_numpy()
19 mean_dt = np.mean(data)
20 std_dt = np.std(data)
21 transformasi_dt = (data-mean_dt)/std_dt
22 norm_trans_dt = []
23 for x in transformasi_dt:
24     x_new = (1-math.exp(-x))/(1+math.exp(-x))
25     norm_trans_dt.append(x_new)
26
27 sns.histplot(norm_trans_dt, bins=20, kde=True)
28 plt.xlabel('Sigmoid Normalisasi ')
29 plt.axvline(np.mean(norm_trans_dt), color='r', linestyle='dashed', linewidth=2)
30 plt.ylabel('Banyaknya data')
31 plt.title("Sebaran Data X setelah normalisasi Sigmoid ")
32 plt.show()
```

Sigmoid Normalisasi

Sebaran Data X setelah normalisasi Sigmoid



Normalisasi Softmax

$$x_{new} = \frac{\exp(x)}{\sum_{i=1}^n \exp(x_i)} \quad (7.6)$$

dimana:

x_{new} : nilai baru hasil normalisasi data

x_i : nilai pengamatan x sebelum normalisasi ke- i

Normalisasi Softmax (2)

No	Data
1	3
2	5
3	5
4	8
5	9
6	12
7	12
8	13
9	15
10	16
11	17
12	19
13	22
14	24
15	25
16	135

$$x_{1_new} = \frac{\exp(3)}{\exp(3) + \exp(5) + \dots + \exp(25) + \exp(135)} = 4.711 e - 58$$

$$x_{2_new} = \frac{\exp(5)}{\exp(3) + \exp(5) + \dots + \exp(25) + \exp(135)} = 3.4811 e - 57$$

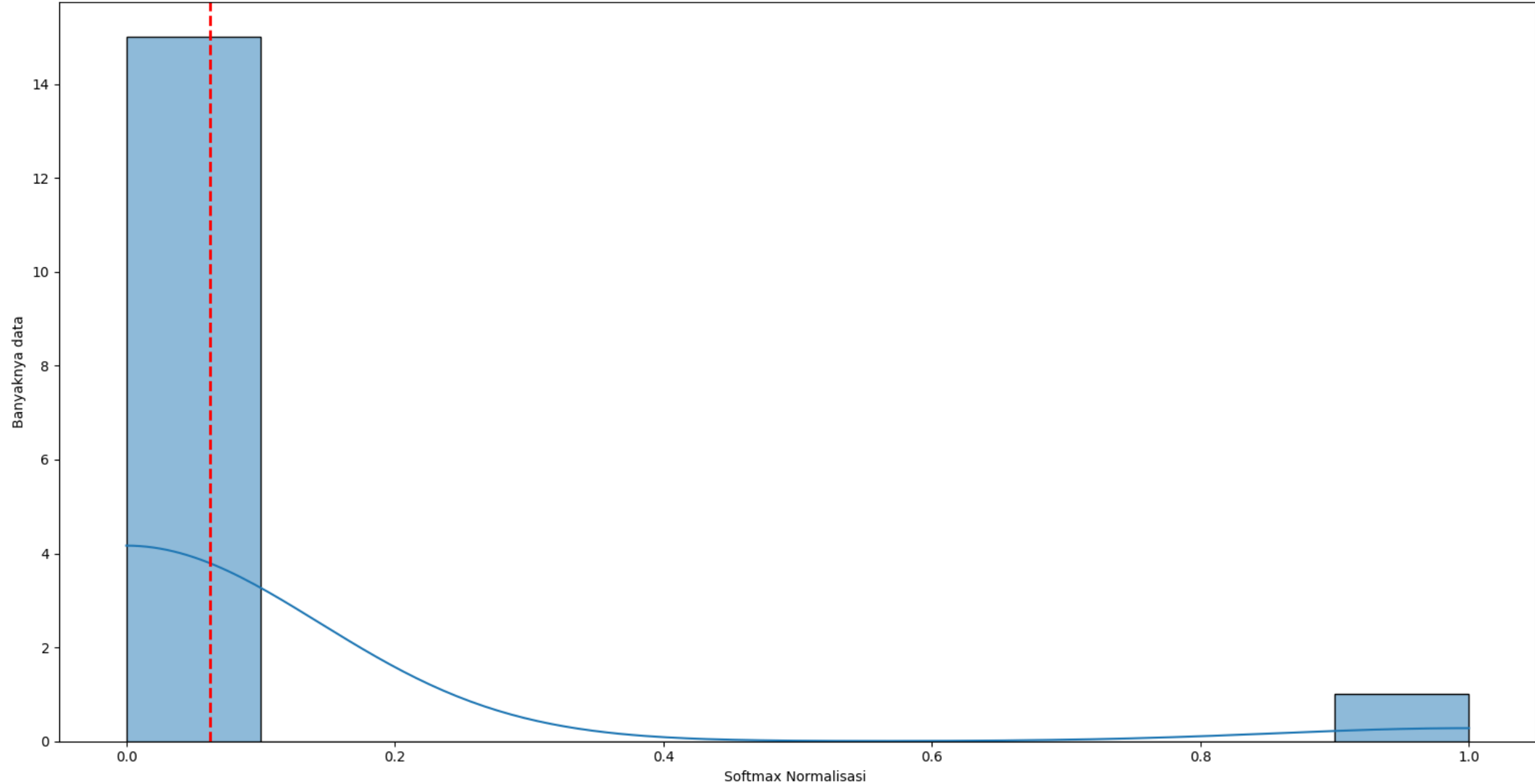
No	Data	exp(x)	x_new
1	3	20.08553692	4.71117E-58
2	5	148.4131591	3.48111E-57
3	5	148.4131591	3.48111E-57
4	8	2980.957987	6.99199E-56
5	9	8103.083928	1.90062E-55
6	12	162754.7914	3.8175E-54
7	12	162754.7914	3.8175E-54
8	13	442413.392	1.0377E-53
9	15	3269017.372	7.66765E-53
10	16	8886110.521	2.08428E-52
11	17	24154952.75	5.66567E-52
12	19	178482301	4.18639E-51
13	22	3584912846	8.4086E-50
14	24	26489122130	6.21316E-49
15	25	72004899337	1.68891E-48
16	135	4.26339E+58	1

Implementasi python

```
1 import matplotlib.pyplot as plt
2 import numpy as np
3 import seaborn as sns
4 import pandas as pd
5
6 data = [3, 5, 5, 8, 9, 12, 12, 13, 15, 16, 17, 19, 22, 24, 25, 135]
7 myData = pd.DataFrame(data)
8 myData.columns = ["X"]
9 print(myData)
10 sns.histplot(myData, bins=20, kde=True)
11 plt.axvline(np.mean(data), color='r', linestyle='dashed', linewidth=2)
12 plt.xticks(range(0,140,5))
13 plt.xlabel('Data X')
14 plt.ylabel('Banyaknya data')
15 plt.title(" Sebaran Data X sebelum normalisasi z score ")
16 plt.show()
17
18 data = myData["X"].to_numpy()
19 trans_dt_softmax = np.exp(data)/(np.sum(np.exp(data), axis=0))
20 sns.histplot(norm_trans_dt, bins=10, kde=True)
21 plt.xlabel('Softmax Normalisasi ')
22 plt.axvline(np.mean(trans_dt_softmax), color='r', linestyle='dashed', linewidth=2)
23 plt.ylabel('Banyaknya data')
24 plt.title("Sebaran Data X setelah normalisasi Softmax ")
25 plt.show()
```

Softmax Normalisasi

Sebaran Data X setelah normalisasi Softmax



Standarisasi

- Sedangkan standarisasi, adalah tehnik lain dalam melakukan perubahan skala, dimana data yang dimiliki akan diubah sehingga memiliki **rata rata = 0 (terpusat) dan standar deviasi = 1**
- Standarisasi sangat membantu ketika data tersebut **tidak terdistribusi** normal secara gaussian
- Standarisasi tidak memiliki jangkauan nilai sebagaimana normalisasi yang memiliki nilai 0–1
- Standarisasi tidak mengubah data outlier.
- Standarisasi dilakukan pada data yang memang belum **mengikuti distribusi normal**
- Untuk 'menormalkan' data, bisa dilakukan dengan **metode transformasi, bukan scaling.**



Tekni Standarisasi Data

- ▶ Z-score
 - ▶ Devide Standardeviasi
 - ▶ 0-1 scaling
 - ▶ Devide range
- 

Standarisasi Z-score

No	Data	x-mean	(x-mean) ²	z_score	z_score - mean_z_score	(z_score - mean_z_score) ²
1	3	-18.250	333.0625	-0.6067134	-0.607	0.368101126
2	5	-16.250	264.0625	-0.5402242	-0.540	0.291842233
3	5	-16.250	264.0625	-0.5402242	-0.540	0.291842233
4	8	-13.250	175.5625	-0.4404905	-0.440	0.194031913
5	9	-12.250	150.0625	-0.407246	-0.407	0.165849278
6	12	-9.250	85.5625	-0.3075123	-0.308	0.094563791
7	12	-9.250	85.5625	-0.3075123	-0.308	0.094563791
8	13	-8.250	68.0625	-0.2742677	-0.274	0.075222767
9	15	-6.250	39.0625	-0.2077786	-0.208	0.043171928
10	16	-5.250	27.5625	-0.174534	-0.175	0.030462112
11	17	-4.250	18.0625	-0.1412894	-0.141	0.019962699
12	19	-2.250	5.0625	-0.0748003	-0.075	0.005595082
13	22	0.750	0.5625	0.0249334	0.025	0.000621676
14	24	2.750	7.5625	0.0914226	0.091	0.008358085
15	25	3.750	14.0625	0.1246671	0.125	0.015541894
16	135	113.750	12939.06	3.7815697	3.782	14.30026939

min
max
means
sum
std

3
135

21.250

14477
30.0801

0.000

16
1

$$z_{1_score} = \frac{3 - 21.25}{30.08} = -0.607$$

$$z_{2_score} = \frac{5 - 21.25}{30.08} = -0.540$$

➤ Jadi distribusi normal → Standarisasi

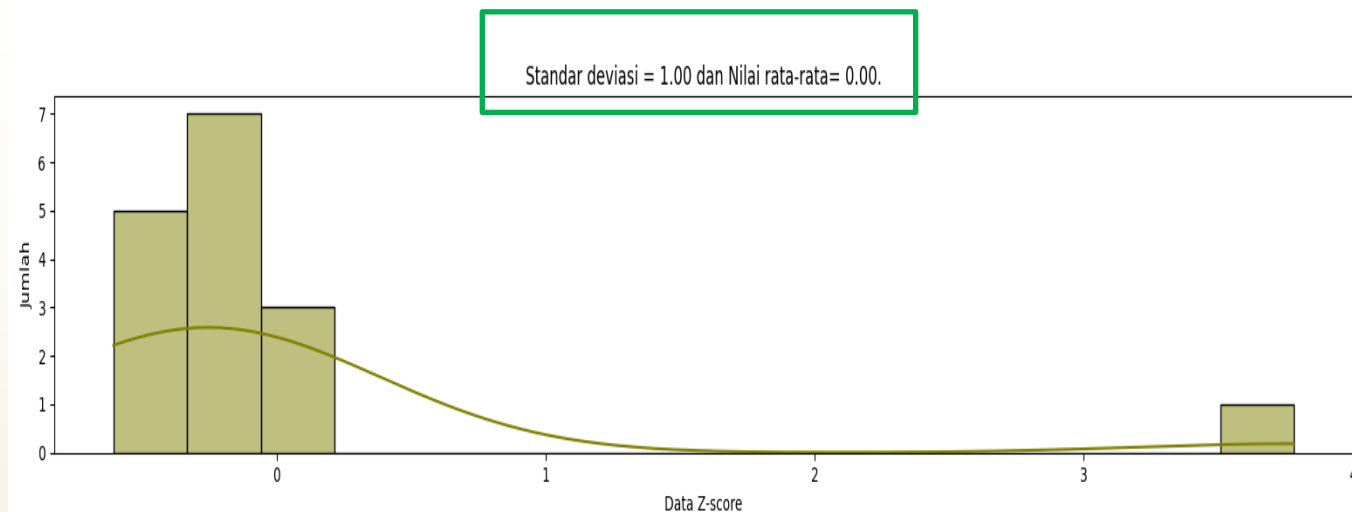
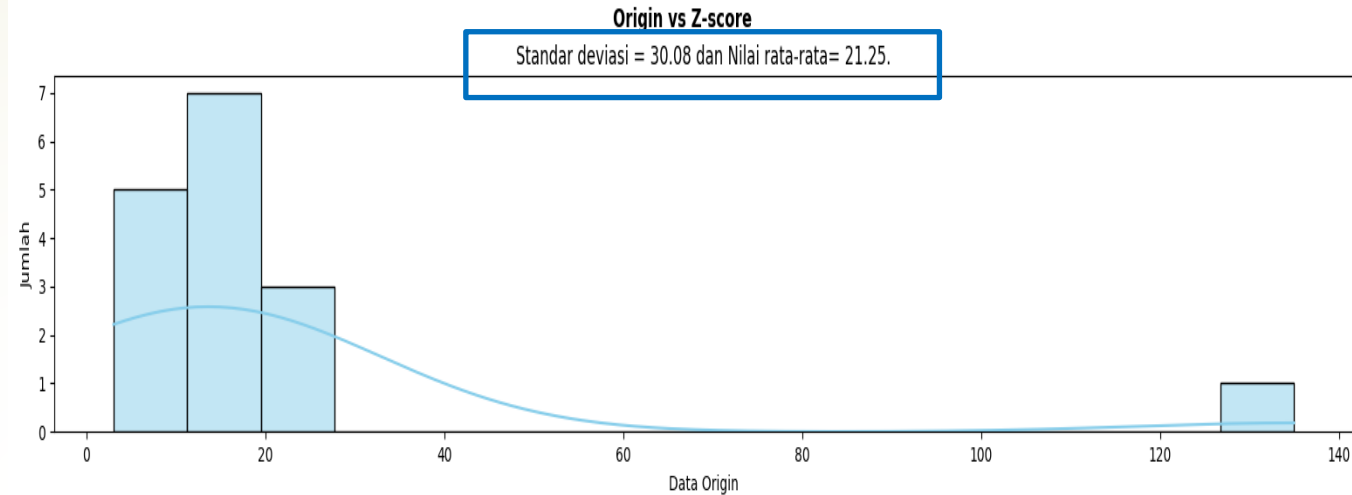
$$x_{new} = \frac{x - \bar{x}}{\delta}$$

data

x : nilai pengamatan x sebelum normalisasi

Implementasi python

```
1 import matplotlib.pyplot as plt
2 import numpy as np
3 import seaborn as sns
4 import pandas as pd
5
6 data = [3, 5, 5, 8, 9, 12, 12, 13, 15, 16, 17, 19, 22, 24, 25, 135]
7 myData = pd.DataFrame(data)
8 myData.columns = ["X"]
9 print(myData)
10 myMean = np.mean(data)
11 myStd = np.std(data)
12 data_z_score = (data-myMean)/myStd
13 myData1 = pd.DataFrame(data)
14 myData_Z_Score = pd.DataFrame(data_z_score)
15 myPD = pd.concat([myData1, myData_Z_Score], axis="columns")
16 myPD.columns = ["Origin", "Z-score"]
17
18 fig, axs = plt.subplots(2, figsize=(4, 4))
19 fig.suptitle("Origin vs Z-score", x=0.5, y=0.93, fontsize=12, fontweight="bold")
20 fig.subplots_adjust(hspace=0.4, wspace=0.4)
21 std_origin = np.std(myPD["Origin"])
22 mean_origin = np.mean(myPD["Origin"])
23 title_origin = 'Standar deviasi = %.2f dan Nilai rata-rata= %.2f.' % (std_origin,
24                               mean_origin)
25 sns.histplot(data=myPD["Origin"], kde=True, color="skyblue",
26              ax=axs[0]).set(title=title_origin)
27 axs[0].set(xlabel='Data Origin', ylabel='Jumlah')
28 std_z_score = np.std(myPD["Z-score"])
29 mean_z_score = np.mean(myPD["Z-score"])
30 title_std = 'Standar deviasi = %.2f dan Nilai rata-rata= %.2f.' % (std_z_score,
31                               mean_z_score)
32 sns.histplot(data=myPD["Z-score"], kde=True, color="olive",
33              ax=axs[1]).set(title=title_std)
34 axs[1].set(xlabel='Data Z-score', ylabel='Jumlah')
35 plt.show()
```



Standarisasi devide Standar deviasi

No	Data	x-mean	(x-mean) ²	X _{new}	x _{new} - mean_x _{new}	(x _{new} - mean_x _{new}) ²
1	3	-18.250	333.0625	0.0997337	-0.607	0.368101126
2	5	-16.250	264.0625	0.1662228	-0.540	0.291842233
3	5	-16.250	264.0625	0.1662228	-0.540	0.291842233
4	8	-13.250	175.5625	0.2659566	-0.440	0.194031913
5	9	-12.250	150.0625	0.2992011	-0.407	0.165849278
6	12	-9.250	85.5625	0.3989348	-0.308	0.094563791
7	12	-9.250	85.5625	0.3989348	-0.308	0.094563791
8	13	-8.250	68.0625	0.4321794	-0.274	0.075222767
9	15	-6.250	39.0625	0.4986685	-0.208	0.043171928
10	16	-5.250	27.5625	0.5319131	-0.175	0.030462112
11	17	-4.250	18.0625	0.5651577	-0.141	0.019962699
12	19	-2.250	5.0625	0.6316468	-0.075	0.005595082
13	22	0.750	0.5625	0.7313805	0.025	0.000621676
14	24	2.750	7.5625	0.7978697	0.091	0.008358085
15	25	3.750	14.0625	0.8311142	0.125	0.015541894
16	135	113.750	12939.0625	4.4880168	3.782	14.30026939

min
max
means
sum
std

3
135
21.250

14477
30.0801

0.706

16
1

0. Standar deviasi

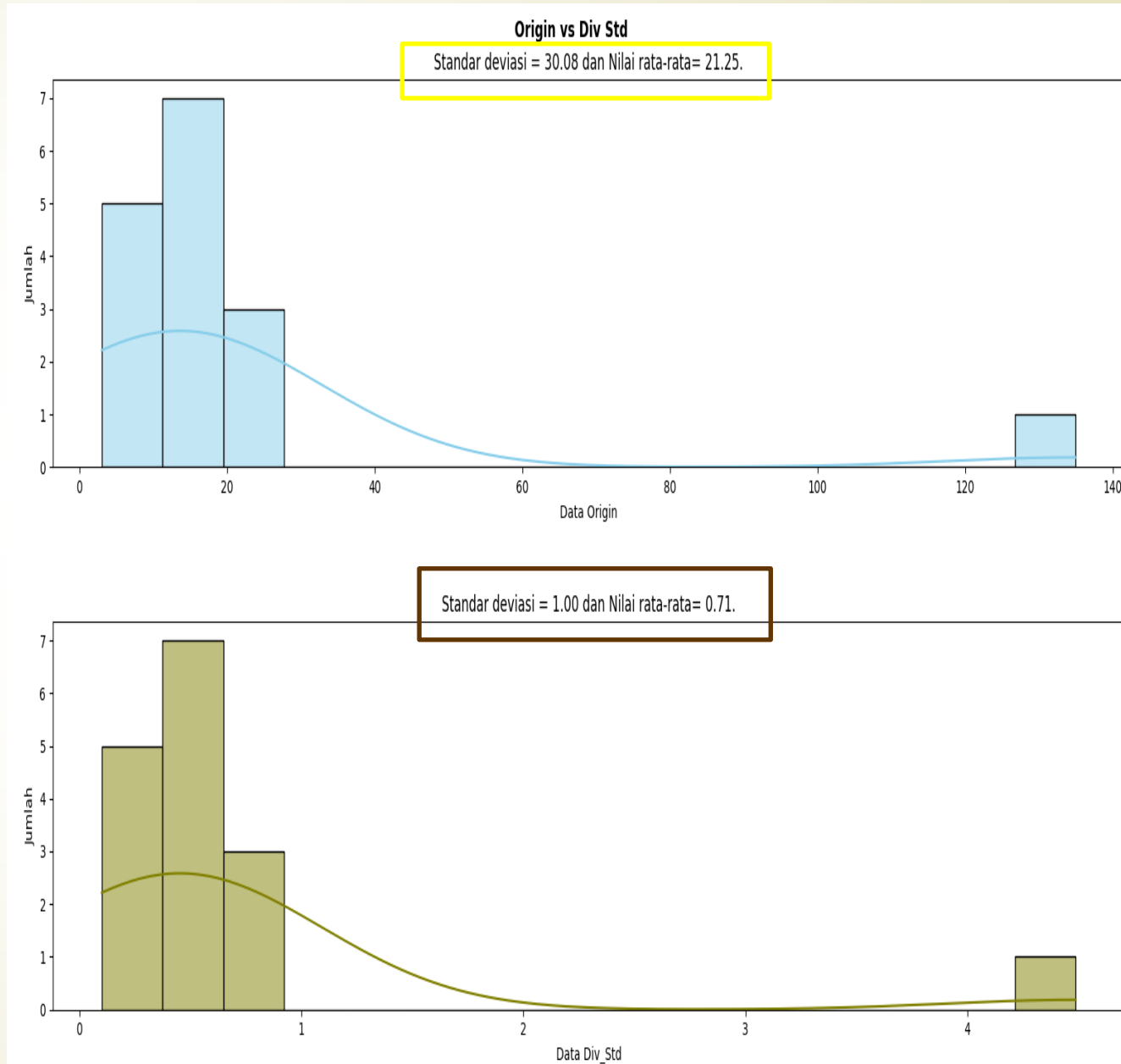
$$x_{1_new} = \frac{3}{30.08} = 0.0997$$

$$x_{new} = \frac{x}{\delta}$$

x : nilai pengamatan x sebelum normalisasi

Implementasi python

```
1 import matplotlib.pyplot as plt
2 import numpy as np
3 import seaborn as sns
4 import pandas as pd
5
6 data = [3, 5, 5, 8, 9, 12, 12, 13, 15, 16, 17, 19, 22, 24, 25, 135]
7 myData = pd.DataFrame(data)
8 myData.columns = ["X"]
9 print(myData)
10 myMean = np.mean(data)
11 myStd = np.std(data)
12 data_div = (data)/myStd
13 myData1 = pd.DataFrame(data)
14 myData_Div = pd.DataFrame(data_div)
15 myPD = pd.concat([myData1, myData_Div], axis="columns")
16 myPD.columns = ["Origin", "Div_Std"]
17
18 fig, axs = plt.subplots(2, figsize=(4, 4))
19 fig.suptitle("Origin vs Div Std", x=0.5, y=0.93, fontsize=12, fontweight="bold")
20 fig.subplots_adjust(hspace=0.4, wspace=0.4)
21 std_origin = np.std(myPD["Origin"])
22 mean_origin = np.mean(myPD["Origin"])
23 title_origin = 'Standar deviasi = %.2f dan Nilai rata-rata= %.2f' % (std_origin,
24                             mean_origin)
25 sns.histplot(data=myPD["Origin"], kde=True, color="skyblue",
26              ax=axs[0]).set(title=title_origin)
27 axs[0].set(xlabel='Data Origin', ylabel='Jumlah')
28 std_div = np.std(myPD["Div_Std"])
29 mean_div = np.mean(myPD["Div_Std"])
30 title_std = 'Standar deviasi = %.2f dan Nilai rata-rata= %.2f' % (std_z_score,
31                             mean_z_score)
32 sns.histplot(data=myPD["Div_Std"], kde=True, color="olive",
33              ax=axs[1]).set(title=title_std)
34 axs[1].set(xlabel='Data Div_Std', ylabel='Jumlah')
35 plt.show()
```



Standarisasi 0-1 skala

No	Data	x-mean	(x-mean) ²	X _{new}	z _{new} - mean _{new}	(z _{new} - mean _{new}) ²
1	3	-18.250	333.0625	0	-0.138	0.019115157
2	5	-16.250	264.0625	0.0151515	-0.123	0.015155102
3	5	-16.250	264.0625	0.0151515	-0.123	0.015155102
4	8	-13.250	175.5625	0.0378788	-0.100	0.010075901
5	9	-12.250	150.0625	0.0454545	-0.093	0.008612402
6	12	-9.250	85.5625	0.0681818	-0.070	0.004910612
7	12	-9.250	85.5625	0.0681818	-0.070	0.004910612
8	13	-8.250	68.0625	0.0757576	-0.063	0.00390625
9	15	-6.250	39.0625	0.0909091	-0.047	0.002241879
10	16	-5.250	27.5625	0.0984848	-0.040	0.00158187
11	17	-4.250	18.0625	0.1060606	-0.032	0.001036645
12	19	-2.250	5.0625	0.1212121	-0.017	0.000290548
13	22	0.750	0.5625	0.1439394	0.006	3.22831E-05
14	24	2.750	7.5625	0.1590909	0.021	0.000434028
15	25	3.750	14.0625	0.1666667	0.028	0.000807076
16	135	113.750	12939.06	1	0.862	0.742600006

$$x_{1_new} = \frac{3 - 3}{135} = 0$$

$$x_{2_new} = \frac{5 - 3}{135} = 0.015$$

$$x_{new} = \frac{x - x_{min}}{x_{max} - x_{min}}$$

min
max
means
sum
std

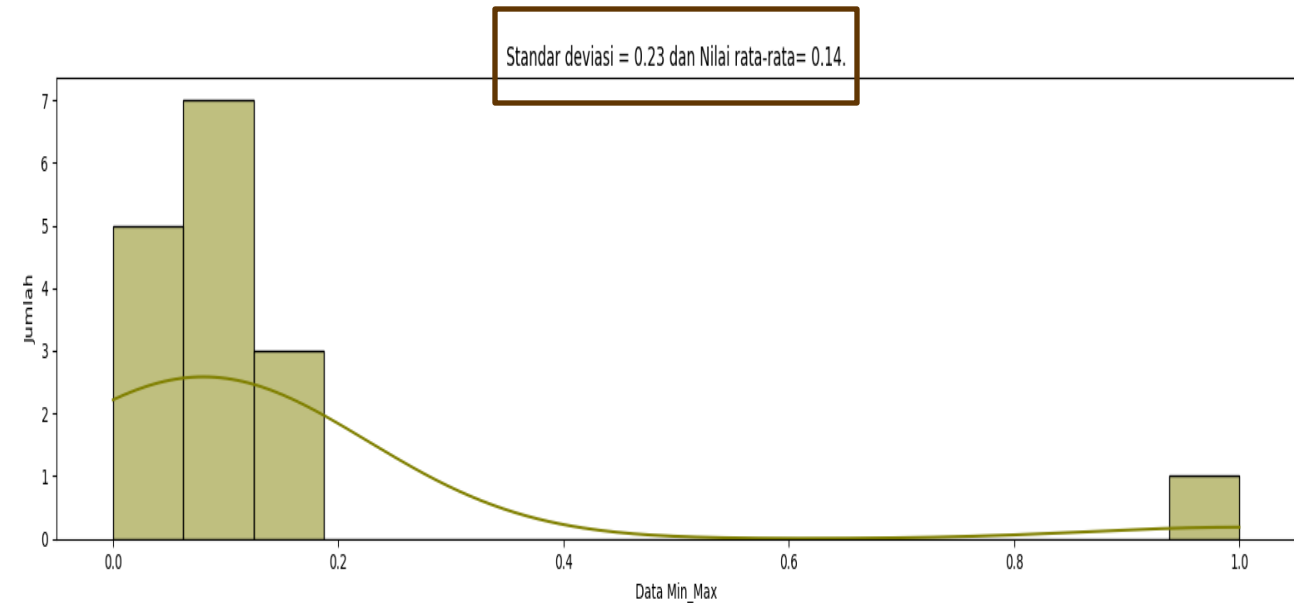
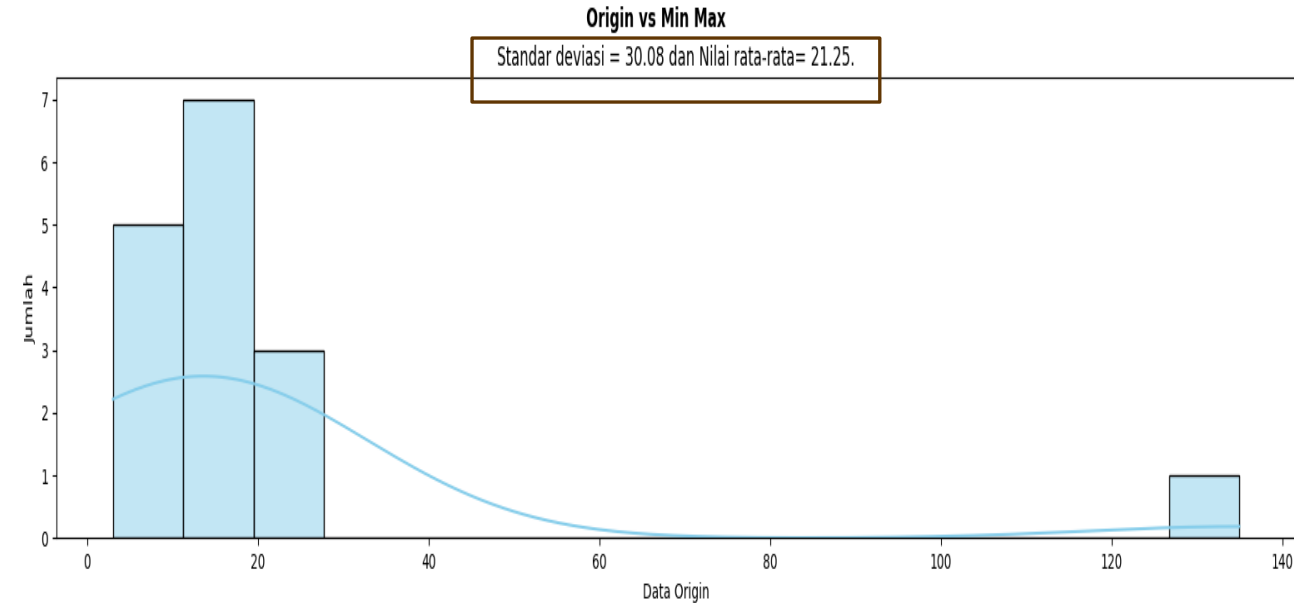
3
135
21.250

0.138
14477
30.0801

0.830865473
0.227879556

Implementasi python

```
1 import matplotlib.pyplot as plt
2 import numpy as np
3 import seaborn as sns
4 import pandas as pd
5
6 data = [3, 5, 5, 8, 9, 12, 12, 13, 15, 16, 17, 19, 22, 24, 25, 135]
7 myData = pd.DataFrame(data)
8 myData.columns = ["X"]
9 print(myData)
10 my_min = np.min(data)
11 my_max = np.max(data)
12 data_min_max = (data-my_min)/(my_max - my_min)
13 myData1 = pd.DataFrame(data)
14 myData_MinMax = pd.DataFrame(data_min_max)
15 myPD = pd.concat([myData1, myData_MinMax], axis="columns")
16 myPD.columns = ["Origin", "Min_Max"]
17
18 fig, axs = plt.subplots(2, figsize=(4, 4))
19 fig.suptitle("Origin vs Min Max", x=0.5, y=0.93, fontsize=12, fontweight="bold")
20 fig.subplots_adjust(hspace=0.4, wspace=0.4)
21 std_origin = np.std(myPD["Origin"])
22 mean_origin = np.mean(myPD["Origin"])
23 title_origin = 'Standar deviasi = %.2f dan Nilai rata-rata= %.2f.' % (std_origin,
24                               mean_origin)
25 sns.histplot(data=myPD["Origin"], kde=True, color="skyblue",
26              ax=axs[0]).set(title=title_origin)
27 axs[0].set(xlabel='Data Origin', ylabel='Jumlah')
28 std_MinMax = np.std(myPD["Min_max"])
29 mean_MinMax = np.mean(myPD["Min_Max"])
30 title_std = 'Standar deviasi = %.2f dan Nilai rata-rata= %.2f.' % (std_MinMax,
31                               mean_MinMax)
32 sns.histplot(data=myPD["Min_Max"], kde=True, color="olive",
33              ax=axs[1]).set(title=title_std)
34 axs[1].set(xlabel='Data Min_Max', ylabel='Jumlah')
35 plt.show()
```



Standarisasi dengan pembagian interval

No	Data	x-mean	(x-mean) ²	x_{new}	$x_{new} - \text{mean}_{x_{new}}$	$(x_{new} - \text{mean}_{x_{new}})^2$
1	3	-18.250	333.0625	0.0227273	-0.138	0.019115157
2	5	-16.250	264.0625	0.0378788	-0.123	0.015155102
3	5	-16.250	264.0625	0.0378788	-0.123	0.015155102
4	8	-13.250	175.5625	0.0606061	-0.100	0.010075901
5	9	-12.250	150.0625	0.0681818	-0.093	0.008612402
6	12	-9.250	85.5625	0.0909091	-0.070	0.004910612
7	12	-9.250	85.5625	0.0909091	-0.070	0.004910612
8	13	-8.250	68.0625	0.0984848	-0.063	0.00390625
9	15	-6.250	39.0625	0.1136364	-0.047	0.002241879
10	16	-5.250	27.5625	0.1212121	-0.040	0.00158187
11	17	-4.250	18.0625	0.1287879	-0.032	0.001036645
12	19	-2.250	5.0625	0.1439394	-0.017	0.000290548
13	22	0.750	0.5625	0.1666667	0.006	3.22831E-05
14	24	2.750	7.5625	0.1818182	0.021	0.000434028
15	25	3.750	14.0625	0.1893939	0.028	0.000807076
16	135	113.750	12939.06	1.0227273	0.862	0.742600006

$$x_{1_new} = \frac{3}{135-3} = 0.02272$$

$$x_{new} = \frac{x}{x_{max} - x_{min}}$$

min
max
means
sum
std

3
135
21.250

14477
30.0801

0.161

0.830865473
0.227879556

baru hasil normalisasi data

amatan x sebelum normalisasi

absolut minimal dari x

absolut maksimal dari x

Implementasi python

```
1 import matplotlib.pyplot as plt
2 import numpy as np
3 import seaborn as sns
4 import pandas as pd
5
6 data = [3, 5, 5, 8, 9, 12, 12, 13, 15, 16, 17, 19, 22, 24, 25, 135]
7 myData = pd.DataFrame(data)
8 myData.columns = ["X"]
9 print(myData)
10 my_min = np.min(data)
11 my_max = np.max(data)
12 data_div_range = (data)/(my_max - my_min)
13 myData1 = pd.DataFrame(data)
14 myData_DivRange = pd.DataFrame(data_div_range)
15 myPD = pd.concat([myData1, myData_DivRange], axis="columns")
16 myPD.columns = ["Origin", "Div_Range"]
17
18 fig, axs = plt.subplots(2, figsize=(4, 4))
19 fig.suptitle("Origin vs Div Range", x=0.5, y=0.93, fontsize=12,
20 fontweight="bold")
21 fig.subplots_adjust(hspace=0.4, wspace=0.4)
22 std_origin = np.std(myPD["Origin"])
23 mean_origin = np.mean(myPD["Origin"])
24 title_origin = 'Standar deviasi = %.2f dan Nilai rata-rata= %.2f.' % (std_origin,
25 mean_origin)
26 sns.histplot(data=myPD["Origin"], kde=True, color="skyblue",
27 ax=axs[0]).set(title=title_origin)
28 axs[0].set(xlabel='Data Origin', ylabel='Jumlah')
29 std_DivRange = np.std(myPD["Div_Range"])
30 mean_DivRange = np.mean(myPD["Div_Range"])
31 title_std = 'Standar deviasi = %.2f dan Nilai rata-rata= %.2f.' % (std_DivRange,
32 mean_DivRange)
33 sns.histplot(data=myPD["Div_Range"], kde=True, color="olive",
34 ax=axs[1]).set(title=title_std)
35 axs[1].set(xlabel='Data Div_Range', ylabel='Jumlah')
36 plt.show()
```

